UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación, Sede Sololá.
Ingeniero Wilmer Orlando Xeron
Base de Datos 1 I Sec. “A”



Fase 1: Arquitectura de datos Corporativa

Cristhian Cesar Felipe Mucún Queché
2290-17-17851

Panajachel 30 de septiembre de 2026

```
-- Autor: Cristhian Mucun | Carne: 2290-17-17851

-- ============================================
USE master;

--Si la base no existe, no intenta eliminarla.
IF DB_ID('ERP_Corporativo') IS NOT NULL --Encuentra una base llamada ERP_Corporativo, entonces
        DROP DATABASE ERP_Corporativo; --elimina completamente la base de datos.

-- Simulamos la creacion de Filegroups para el particionamiento físico
CREATE DATABASE ERP_Corporativo;
USE ERP_Corporativo;

--SEGURIDAD y SCHEMAS
CREATE LOGIN ErpAdmin WITH PASSWORD = 'Sys@ErpPassword2026!';
CREATE USER ErpAdmin FOR LOGIN ErpAdmin;
ALTER ROLE db_owner ADD MEMBER ErpAdmin;


CREATE SCHEMA gbl; -- Globales y Geografía
CREATE SCHEMA hr; -- Recursos Humanos y Nómina
CREATE SCHEMA inv; -- Inventarios y Cadena de Suministros
CREATE SCHEMA pos; -- Punto de Venta y Clientes
CREATE SCHEMA cont; -- Contabilidad y Finanzas

-- PARTICIONAMIENTO
CREATE PARTITION FUNCTION pf_DepartamentosGT (INT)
AS RANGE LEFT FOR VALUES
(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22);

CREATE PARTITION SCHEME ps_DepartamentosGT
AS PARTITION pf_DepartamentosGT
ALL TO ([PRIMARY]);

-- SECUENCIAS
CREATE SEQUENCE cont.Seq_PartidasContables
```

```
START WITH 1
INCREMENT BY 1;

CREATE SEQUENCE pos.Seq_Facturas
START WITH 1
INCREMENT BY 1;

--DOMINIO GLOBAL
CREATE TABLE glb_Paises (
    IdPais INT IDENTITY(1,1) PRIMARY KEY,
    NombrePais VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE glb_Departamentos (
    IdDepto INT IDENTITY(1,1) PRIMARY KEY,
    IdPais INT NOT NULL REFERENCES glb_Paises(IdPais),
    NombreDepto VARCHAR(100) NOT NULL
);

CREATE TABLE glb_Municipios (
    IdMunicipio INT IDENTITY(1,1) PRIMARY KEY,
    IdDepto INT NOT NULL REFERENCES
glb_Departamentos(IdDepto),
    NombreMunicipio VARCHAR(100) NOT NULL
);

CREATE TABLE glb_Monedas (
    IdMoneda INT IDENTITY(1,1) PRIMARY KEY,
    CodigoISO CHAR(3) NOT NULL UNIQUE,
    NombreMoneda VARCHAR(50) NOT NULL,
    TasaCambioBase DECIMAL(10,4) NOT NULL DEFAULT 1.0000
);

CREATE TABLE glb_Empresas (
    IdEmpresa INT IDENTITY(1,1) PRIMARY KEY,
    NIT VARCHAR(20) NOT NULL UNIQUE,
    RazonSocial VARCHAR(150) NOT NULL,
    IdMonedaLocal INT NOT NULL REFERENCES
glb_Monedas(IdMoneda),
    Activo BIT DEFAULT 1
```

```
);

CREATE TABLE glb_Sucursales (
    IdSucursal INT IDENTITY(1,1) PRIMARY KEY,
    IdEmpresa INT NOT NULL REFERENCES
glb_Empresas(IdEmpresa),
    IdMunicipio INT NOT NULL REFERENCES
glb_Municipios(IdMunicipio),
    NombreSucursal VARCHAR(100) NOT NULL,
    EsMatriz BIT DEFAULT 0
);

-- DOMINIO RECURSOS HUMANOS
CREATE TABLE hr_DepartamentosEmpresa (
    IdDeptoEmpresa INT IDENTITY(1,1) PRIMARY KEY,
    NombreDepartamento VARCHAR(100) NOT NULL
);

CREATE TABLE hr_Puestos (
    IdPuesto INT IDENTITY(1,1) PRIMARY KEY,
    NombrePuesto VARCHAR(100) NOT NULL,
    IdPuestoJefe INT NULL,
    SalarioBase DECIMAL(12,2) NOT NULL CHECK (SalarioBase >
0),

    CONSTRAINT FK_Puesto_Jefatura
    FOREIGN KEY (IdPuestoJefe)
    REFERENCES hr_Puestos(IdPuesto)
);

CREATE TABLE hr_Empleados (
    IdEmpleado INT IDENTITY(1,1) PRIMARY KEY,
    IdSucursal INT NOT NULL REFERENCES
glb_Sucursales(IdSucursal),
    IdDeptoEmpresa INT NOT NULL
        REFERENCES hr_DepartamentosEmpresa(IdDeptoEmpresa),
    IdPuesto INT NOT NULL REFERENCES hr_Puestos(IdPuesto),
    DPI VARCHAR(15) NOT NULL UNIQUE,
    NombreCompleto VARCHAR(150) NOT NULL,
    FechaNacimiento DATE NOT NULL,
```

```
    FechaIngreso DATE DEFAULT GETDATE(),
    Estado VARCHAR(20) DEFAULT 'Activo'
        CHECK (Estado IN ('Activo', 'Suspendido', 'Baja'))
);

CREATE TABLE hr_PlanillaMaster (
    IdPlanilla INT IDENTITY(1,1) PRIMARY KEY,
    IdEmpresa INT NOT NULL REFERENCES
glb_Empresas(IdEmpresa),
    Mes INT CHECK (Mes BETWEEN 1 AND 12),
    Anio INT CHECK (Anio > 2000),
    TotalPagado DECIMAL(18,2) DEFAULT 0,
    FechaProceso DATETIME DEFAULT GETDATE(),
    Procesada BIT DEFAULT 0
);

CREATE TABLE hr_PlanillaDetalle (
    IdDetalle INT IDENTITY(1,1) PRIMARY KEY,
    IdPlanilla INT NOT NULL REFERENCES
hr_PlanillaMaster(IdPlanilla),
    IdEmpleado INT NOT NULL REFERENCES
hr_Empleados(IdEmpleado),
    DiasTrabajados INT NOT NULL DEFAULT 30,
    SalarioDevengado DECIMAL(12,2) NOT NULL,
    Bonificaciones DECIMAL(12,2) DEFAULT 0,
    Descuentos DECIMAL(12,2) DEFAULT 0,
    LiquidoRecibir AS
        ((SalarioDevengado + Bonificaciones) - Descuentos)
PERSISTED
);

--DOMINIO CONTABILIDAD
CREATE TABLE cont_Nomenclatura (
    IdCuenta INT IDENTITY(1,1) PRIMARY KEY,
    IdEmpresa INT NOT NULL REFERENCES
glb_Empresas(IdEmpresa),
    CodigoCuenta VARCHAR(20) NOT NULL,
    NombreCuenta VARCHAR(150) NOT NULL,
    IdCuentaPadre INT NULL REFERENCES
cont_Nomenclatura(IdCuenta),
```

```
    TipoCuenta VARCHAR(20)
        CHECK (TipoCuenta IN ('Activo', 'Pasivo', 'Capital', 'Ingreso',
'Gasto')),
    AceptaMovimientos BIT DEFAULT 1
);

CREATE TABLE cont_PartidasMaster (
    IdPartida INT PRIMARY KEY
        DEFAULT NEXT VALUE FOR cont.Seq_PartidasContables,
    IdEmpresa INT NOT NULL REFERENCES
glb_Empresas(IdEmpresa),
    IdSucursal INT NOT NULL REFERENCES
glb_Sucursales(IdSucursal),
    FechaContable DATE DEFAULT GETDATE(),
    Concepto VARCHAR(255) NOT NULL,
    TotalDebe DECIMAL(18,2) DEFAULT 0,
    TotalHaber DECIMAL(18,2) DEFAULT 0,
    Estado VARCHAR(20) DEFAULT 'Cuadrada'
);

CREATE TABLE cont_PartidasDetalle (
    IdDetalle INT IDENTITY(1,1) PRIMARY KEY,
    IdPartida INT NOT NULL
        REFERENCES cont_PartidasMaster(IdPartida)
        ON DELETE CASCADE,
    IdCuenta INT NOT NULL
        REFERENCES cont_Nomenclatura(IdCuenta),
    Debe DECIMAL(18,2) DEFAULT 0,
    Haber DECIMAL(18,2) DEFAULT 0,

    CONSTRAINT CK_DebeHaber
    CHECK (
        Debe >= 0
        AND Haber >= 0
        AND (Debe > 0 OR Haber > 0)
    )
);

--DOMINIO INVENTARIOS
CREATE TABLE inv_Proveedores (
```

```
    IdProveedor INT IDENTITY(1,1) PRIMARY KEY,
    NIT VARCHAR(20) NOT NULL UNIQUE,
    RazonSocial VARCHAR(150) NOT NULL,
    DiasCredito INT DEFAULT 0
);

CREATE TABLE inv_Bodegas (
    IdBodega INT IDENTITY(1,1) PRIMARY KEY,
    IdSucursal INT NOT NULL REFERENCES
glb_Sucursales(IdSucursal),
    NombreBodega VARCHAR(100) NOT NULL
);

CREATE TABLE inv_Categorias (
    IdCategoria INT IDENTITY(1,1) PRIMARY KEY,
    NombreCategoria VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE inv_Productos (
    IdProducto INT IDENTITY(1,1) PRIMARY KEY,
    IdCategoria INT NOT NULL REFERENCES
inv_Categorias(IdCategoria),
    SKU VARCHAR(50) NOT NULL UNIQUE,
    Descripcion VARCHAR(200) NOT NULL,
    CostoPromedio DECIMAL(18,4) DEFAULT 0.0000,
    UltimoCosto DECIMAL(18,4) DEFAULT 0.0000,
    PrecioVentaBase DECIMAL(12,2) NOT NULL CHECK
(PrecioVentaBase > 0),
    ManejaLote BIT DEFAULT 0,
    Activo BIT DEFAULT 1
);

CREATE TABLE inv_MovimientosMaster (
    IdMovimiento INT IDENTITY(1,1) PRIMARY KEY,
    IdBodega INT NOT NULL REFERENCES
inv_Bodegas(IdBodega),
    IdProveedor INT NULL REFERENCES
inv_Proveedores(IdProveedor),
    TipoMovimiento VARCHAR(10)
        CHECK (TipoMovimiento IN ('Ingreso', 'Egreso')),
```

```
    DocumentoRespaldo VARCHAR(50) NOT NULL,
    FechaMovimiento DATETIME DEFAULT GETDATE()
);

CREATE TABLE inv_MovimientosDetalle (
    IdDetalle INT IDENTITY(1,1) PRIMARY KEY,
    IdMovimiento INT NOT NULL
        REFERENCES inv_MovimientosMaster(IdMovimiento)
        ON DELETE CASCADE,
    IdProducto INT NOT NULL REFERENCES
inv_Productos(IdProducto),
    Cantidad DECIMAL(10,2) NOT NULL CHECK (Cantidad > 0),
    CostoUnitario DECIMAL(18,4) NOT NULL CHECK (CostoUnitario
>= 0),
    CostoTotalLinea AS (Cantidad * CostoUnitario) PERSISTED
);

--POS y VENTAS
CREATE TABLE pos_TiposCliente (
    IdTipoCliente INT IDENTITY(1,1) PRIMARY KEY,
    NombreTipo VARCHAR(50) NOT NULL,
    PorcentajeDescuento DECIMAL(5,2)
        DEFAULT 0.00
        CHECK (PorcentajeDescuento BETWEEN 0 AND 100)
);

CREATE TABLE pos_Clientes (
    IdCliente INT IDENTITY(1,1) PRIMARY KEY,
    IdTipoCliente INT NOT NULL
        REFERENCES pos_TiposCliente(IdTipoCliente),
    NIT VARCHAR(15) DEFAULT 'CF',
    NombreCompleto VARCHAR(150) NOT NULL,
    LimiteCredito DECIMAL(12,2)
        DEFAULT 0
        CHECK (LimiteCredito >= 0)
);

CREATE TABLE pos_TiposPago (
    IdTipoPago INT IDENTITY(1,1) PRIMARY KEY,
    NombrePago VARCHAR(50) NOT NULL,
```

```
    RequiereAutorizacion BIT DEFAULT 0
);

CREATE TABLE pos_VentasMaster (
    IdVenta INT NOT NULL,
    IdDepto INT NOT NULL,
    IdSucursal INT NOT NULL
        REFERENCES glb_Sucursales(IdSucursal),
    IdCliente INT NOT NULL
        REFERENCES pos_Clientes(IdCliente),
    IdEmpleadoCajero INT NOT NULL
        REFERENCES hr_Empleados(IdEmpleado),
    FolioFactura INT
        DEFAULT NEXT VALUE FOR pos.Seq_Facturas,
    FechaVenta DATETIME DEFAULT GETDATE(),
    TotalNeto DECIMAL(12,2)
        DEFAULT 0
        CHECK (TotalNeto >= 0),
    TotalDescuentos DECIMAL(12,2)
        DEFAULT 0,
    GranTotal AS
        (TotalNeto - TotalDescuentos) PERSISTED,
    Estado VARCHAR(20) DEFAULT 'Emitida',

    CONSTRAINT CK_VentasMaster_Descuentos
        CHECK (
            TotalDescuentos >= 0
            AND TotalDescuentos <= TotalNeto
        ),
    CONSTRAINT PK_VentasMaster
        PRIMARY KEY CLUSTERED (IdVenta, IdDepto)

) ON ps_DepartamentosGT(IdDepto);

CREATE TABLE pos_VentasDetalle (
    IdDetalle INT IDENTITY(1,1) PRIMARY KEY,
    IdVenta INT NOT NULL,
    IdDepto INT NOT NULL,
    IdProducto INT NOT NULL
        REFERENCES inv_Productos(IdProducto),
```

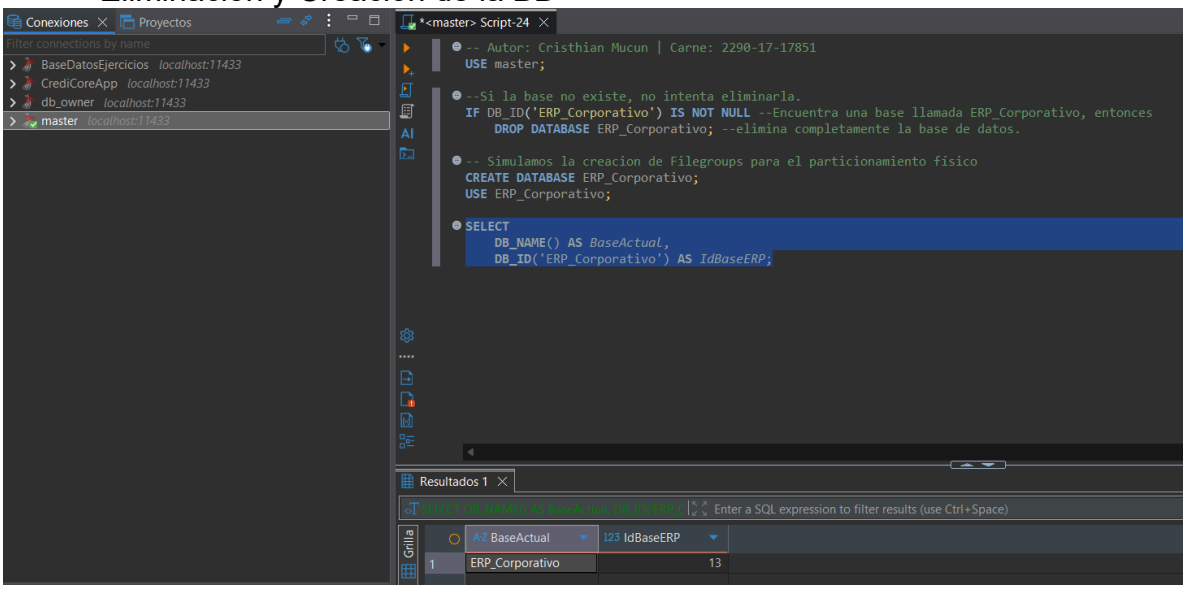
```
    IdBodega INT NOT NULL
        REFERENCES inv_Bodegas(IdBodega),
    Cantidad INT NOT NULL
        CHECK (Cantidad > 0),
    PrecioUnitario DECIMAL(12,2)
        NOT NULL
        CHECK (PrecioUnitario > 0),
    DescuentoLinea DECIMAL(12,2)
        DEFAULT 0,
    SubTotalLinea AS
        ((Cantidad * PrecioUnitario) - DescuentoLinea) PERSISTED,
    CONSTRAINT CK_VentasDetalle_Descuento
        CHECK (
            DescuentoLinea >= 0
            AND DescuentoLinea <= Cantidad * PrecioUnitario
        ),
    CONSTRAINT FK_VentasDetalle_Master
        FOREIGN KEY (IdVenta, IdDepto)
        REFERENCES pos_VentasMaster(IdVenta, IdDepto)
        ON DELETE CASCADE
);

CREATE TABLE pos_PagosVenta (
    IdPago INT IDENTITY(1,1) PRIMARY KEY,
    IdVenta INT NOT NULL,
    IdDepto INT NOT NULL,
    IdTipoPago INT NOT NULL
        REFERENCES pos_TiposPago(IdTipoPago),
    IdMoneda INT NOT NULL
        REFERENCES glb_Monedas(IdMoneda),
    MontoPagado DECIMAL(12,2)
        NOT NULL
        CHECK (MontoPagado > 0),
    Referencia VARCHAR(100) NULL,
    CONSTRAINT FK_PagosVenta_Master
        FOREIGN KEY (IdVenta, IdDepto)
        REFERENCES pos_VentasMaster(IdVenta, IdDepto)
);
```

Análisis crítico empresarial:

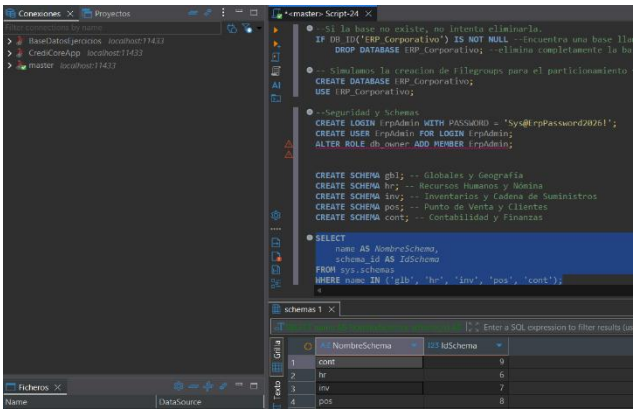
Cimientos de Titanio:

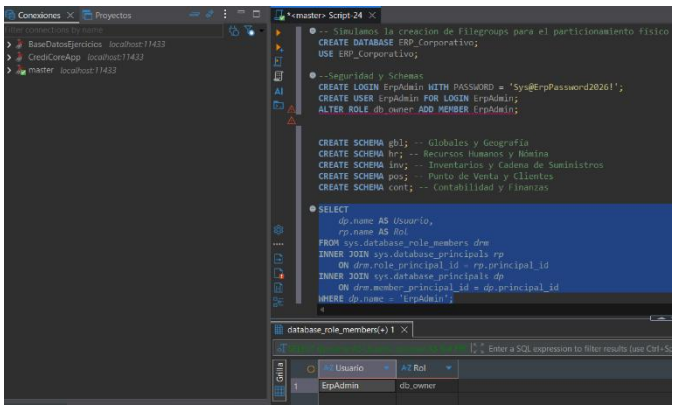
- Eliminación y Creación de la BD



Con este bloque preparamos el entorno principal del ERP, primero se cambia el contexto a la base ‘master’, desde donde se verifica mediante el DB_ID() si ya existe una base llamada ERP_Corporativo. Si existe lo eliminamos para permitir reconstruir el laboratorio desde un estado limpio y posteriormente se crea nuevamente, finalmente se cambia el contexto hacia ERP_Corporativo para asegurar que los siguientes objetos del sistema sean creados dentro de la base correcta, esta estrategia facilita que el despliegue sea repetible durante el desarrollo, sin embargo el uso automático de DROP DATABASE sería riesgoso en un ambiente de producción porque provocaría la pérdida completa de la información.

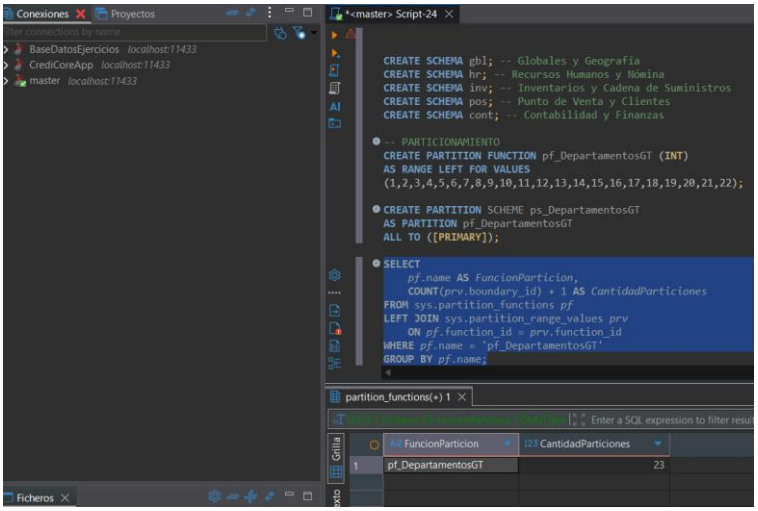
- Seguridad y Schemas.





En este bloque establecemos la seguridad y separación lógica inicial del ERP, se crea el login para permitir la autenticación contra SQL Server y posteriormente se asocia con un usuario dentro de ERP_Corporativo, separando así el acceso al servidor de los permisos específicos sobre la base de datos. El usuario es agregado al rol db_owner para disponer de los permisos necesarios durante la construcción del laboratorio, también se crean los schemas gbl, hr, inv, pos y cont, cuyo objetivo es dividir los objetos del ERP por dominio empresarial y facilitar su administración y control de permisos, en un ambiente de producción seria recomendable aplicar el principio de mínimo privilegio en ligar de otorgar db_owner a una aplicación.

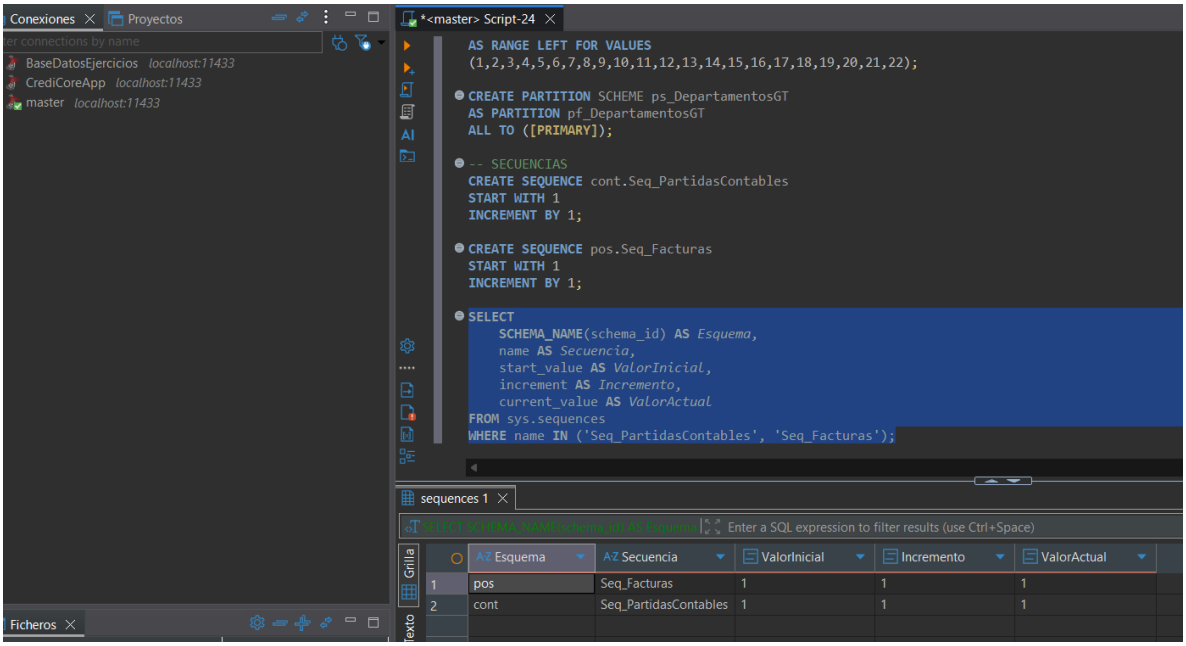
- Particionamientos:



Creamos la infraestructura de particionamiento que posteriormente utilizára la tabla de ventas del ERP, la función pf_DepartametosGT define cómo SQL Server distribuye los valores de los IdDepto entre diferentes particiones mediante RANGE LEFT, mientras que el esquema ps_DepartamentosGT determina dónde serán almacenadas dichas particiones, en este laboratorio todas se asignan al filegroup PRIMARY, por lo que se demuestra la arquitectura lógica de particionamiento sin distribuir físicamente los datos entre distintos discos. Este diseño nos puede

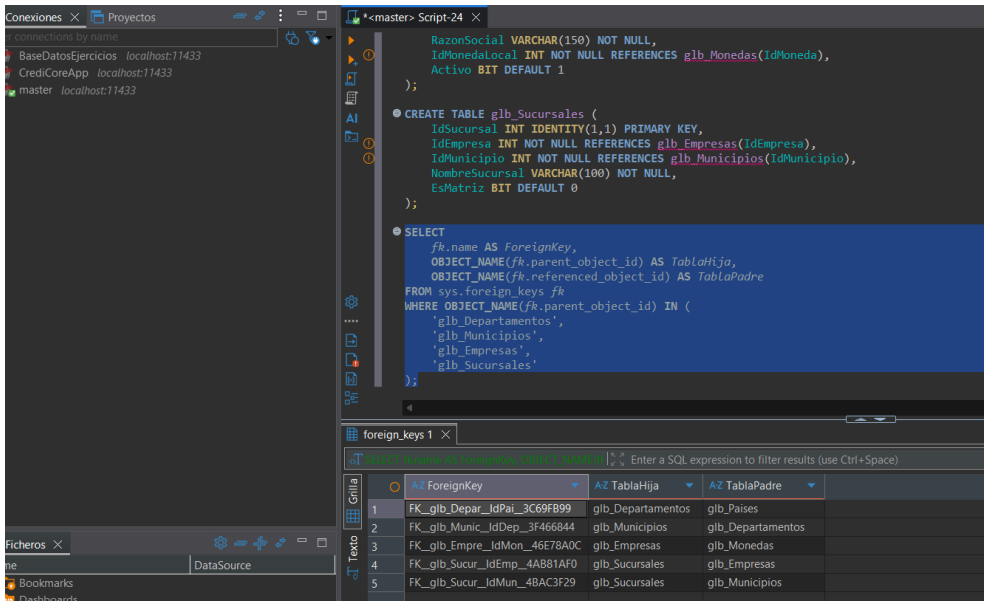
beneficiar tablas de ventas de granvolumen cuando las consultas filtran por la columna de partición, como observación los 22 valores límite definidos generan 23 particiones, debido a que SQL Server siempre crea una partición adicional para los valores que quedan fuera del último límite.

- Secuencias:



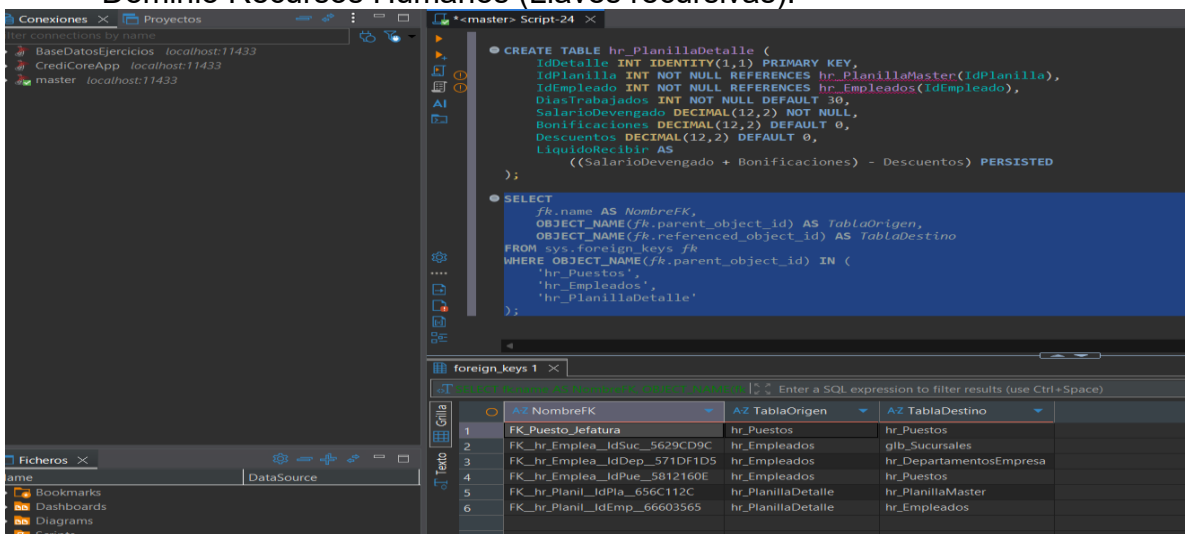
Creamos dos secuencias independientes para controlar la numeración de las partidas contables y las facturas, las secuencias comienzan en 1 y aumentan de uno en uno mediante INCREMENT BY 1, a diferencia de IDENTITY, una secuencia es un objeto independiente que puede ser utilizado por diferentes tablas o procedimientos mediante NEXT VALUE FOR, lo cual permite separar los identificadores internos de los folios empresariales, esta separación facilita el control del documento y evita depender directamente de llaves primarias de una tabla, pero esto no garantiza por si solo que la numeración sea completamente continua.

- Dominio Global (Multiempresa y Geografía).



Construimos el dominio global, centralizando la información geográfica, monetaria y empresarial que será utilizada por los demás módulos, las claves primarias con IDENTITY permiten identificar cada registro de forma única, mientras que restricciones como NOT NULL, UNIQUE y DEFAULT evitan datos incompletos, duplicados o estados iniciales inconsistentes. Las claves foráneas establecen relación entre países, departamentos, municipios, monedas, empresas y sucursales, protegiendo la integridad referencial al impedir que se almacenen registros relacionados con entidades inexistentes.

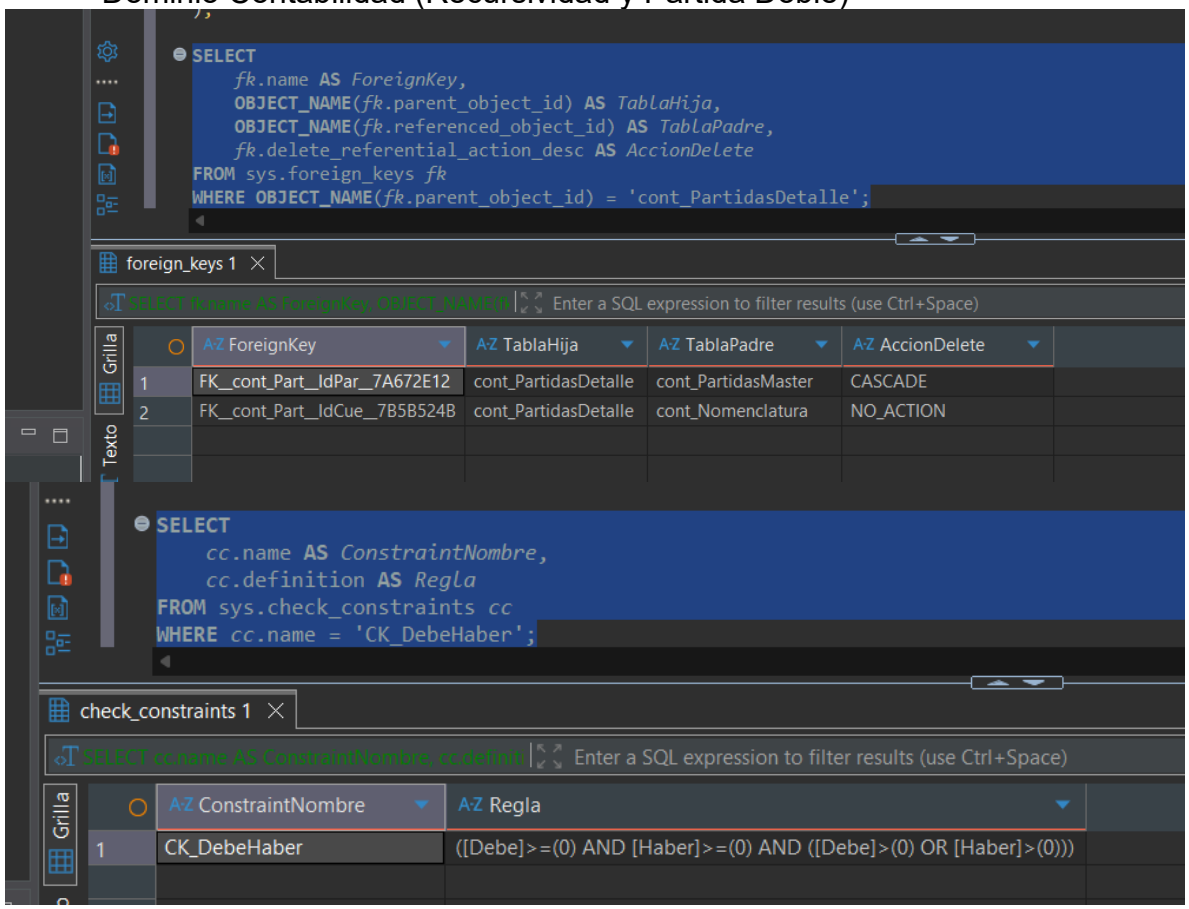
- Dominio Recursos Humanos (Llaves recursivas):



Construimos el módulo de recursos humanos mediante departamentos internos, puestos, empleados y planillas, las tablas de puestos utiliza una clave foránea recursiva que permite representar jerarquías organizacionales, ya que un puesto

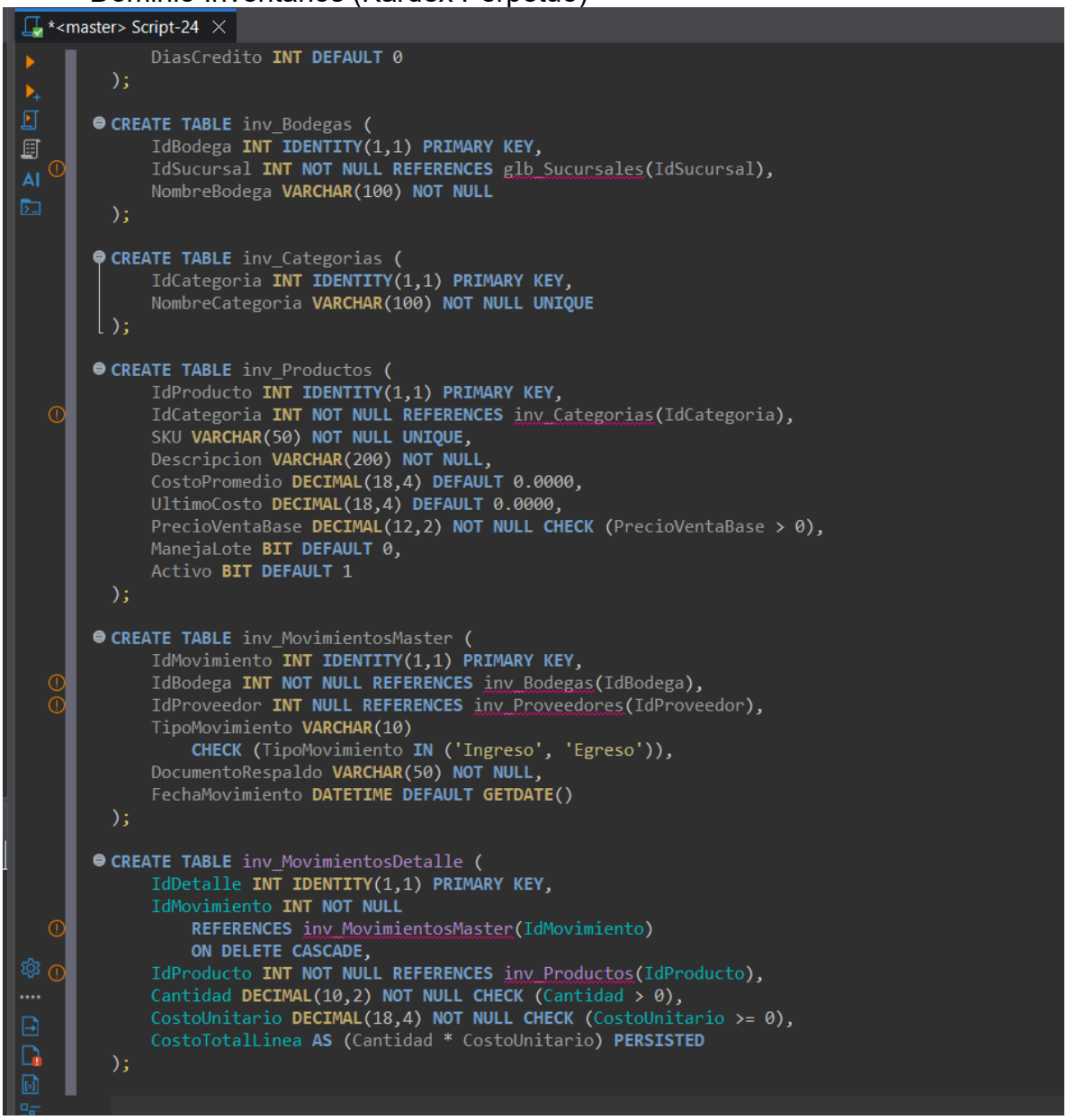
puede depender de otro puesto superior sin duplicar estructuras, la restricciones CHECK protegen reglas como salarios positios, meses válidos y estados permitidos, mientras que las claves foráneas garantizan que cada empleado esté relacionado con una sucursal, departamento y puesto existente, el modelo maestro-detalle de planilla separa la información general del proceso de los valores correspondientes a cada empleado, finalmente LiquidoRecibir se define como una columna calculada persistida, centralizado en el motor la fórmula de salario, bonificaciones y descuentos para reducir inconsistencias entre aplicaciones.

- Dominio Contabilidad (Recursividad y Partida Doble)



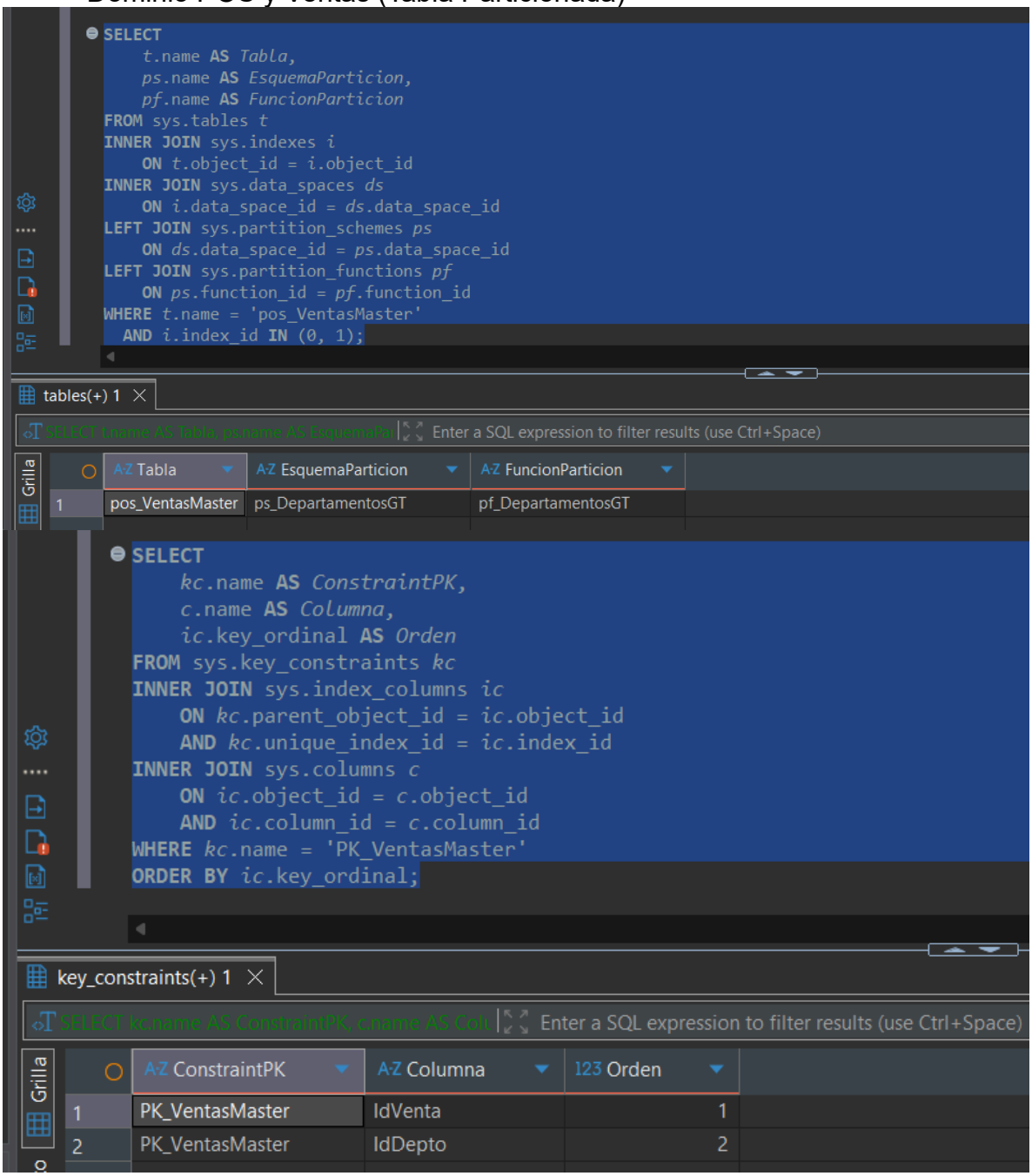
Construye la estructura contable del ERP. La nomenclatura utiliza una clave foránea recursiva para representar cuentas padre e hijas, permitiendo construir jerarquías contables sin duplicar estructuras. Las partidas siguen un modelo maestro-detalle: la cabecera identifica empresa, sucursal, fecha y totales, mientras que el detalle registra las cuentas afectadas en Debe o Haber. La numeración de partidas utiliza la secuencia creada previamente y ON DELETE CASCADE evita detalles huérfanos cuando se elimina una partida maestra, el constraint CK_DebeHaber impide montos negativos y líneas completamente vacías, trasladando parte de la validación contable directamente al motor de base de datos.

- Dominio Inventarios (Kardex Perpetuo)



implementa la estructura base del Kardex perpetuo mediante proveedores, bodegas, categorías, productos y movimientos maestro-detalle. Las claves foráneas garantizan que los movimientos solamente utilicen bodegas, proveedores y productos existentes, mientras que los CHECK evitan cantidades y costos negativos. El detalle utiliza una columna calculada persistida para obtener automáticamente el costo total de cada línea y evita dejar detalles huérfanos. Durante el análisis se agregó StockActual a productos porque la lógica de costeo definida para fases posteriores necesita conocer las existencias disponibles, Estas validaciones trasladan reglas esenciales al motor y reducen la posibilidad de que aplicaciones externas introduzcan datos inválidos.

- Dominio POS y Ventas (Tabla Particionada)



implementa el módulo de clientes, ventas y pagos del ERP. La tabla pos_VentasMaster utiliza IdDepto como columna de particionamiento y una clave primaria compuesta por IdVenta e IdDepto, permitiendo que el índice quede alineado con la estrategia de partición definida previamente. Las tablas de detalle y pagos conservan ambas columnas para mantener la integridad referencial con la venta. La secuencia pos.Seq_Facturas administra independientemente el folio de facturación, mientras que las columnas calculadas obtienen automáticamente el gran total y subtotal de las líneas. También se reforzaron reglas de negocio para

impedir descuentos negativos o superiores al valor de la venta, precios no positivos y límites de crédito inválidos.

Repositorio.
Video.